

Input Kit Documentation

Table of Contents

Introduction

Installation

Getting Started

How to Use

Input Manager Prefab

Joysticks

Joystick Prefab

Input Handler Script

GetFire

GetJump

GetCrouch

GetSprint

GetReload

GetInventory

GetInteraction

GetMap

GetPrevious

GetNext

GetBack

GetPause

GetLeft

GetRight

GetSwipe

GetMove

GetScreenPosition

GetWorldPosition

GetCast2D

GetCast3D

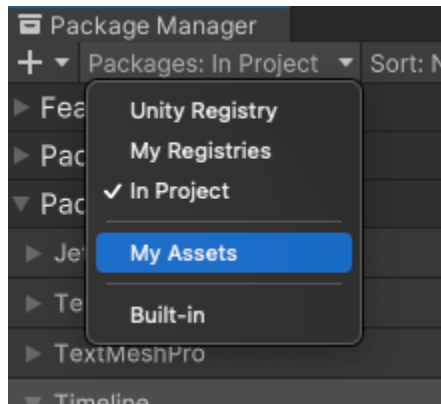
GetCastRect

GetKey<T>

Introduction

Input Kit is a highly versatile input management solution built to simplify input handling across all Unity project types. Designed for developers who want maximum compatibility, clarity, and control, Input Kit abstracts and unifies input from multiple systems—so you can focus on gameplay, not plumbing. Whether you're targeting desktop, console, mobile, or web, Input Kit ensures consistent input behavior, seamless cross-platform deployment, and smooth support for both 2D and 3D interactions.

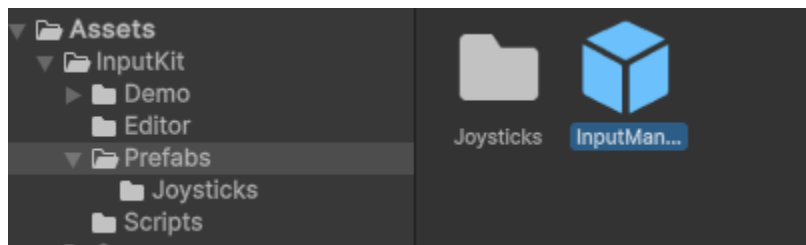
Installation



1. Go to **Window > Package Manager** and open the **Package Manager** window.
2. Select **My Assets** from the **Packages** menu to list any **packages** that you have **purchased**.
3. Select the **Input Kit** from the list of packages. (Try searching for it if it is not listed.)
4. Follow the prompts to complete the installation.

If you are doing this for the first time, [visit here](#).

Getting Started

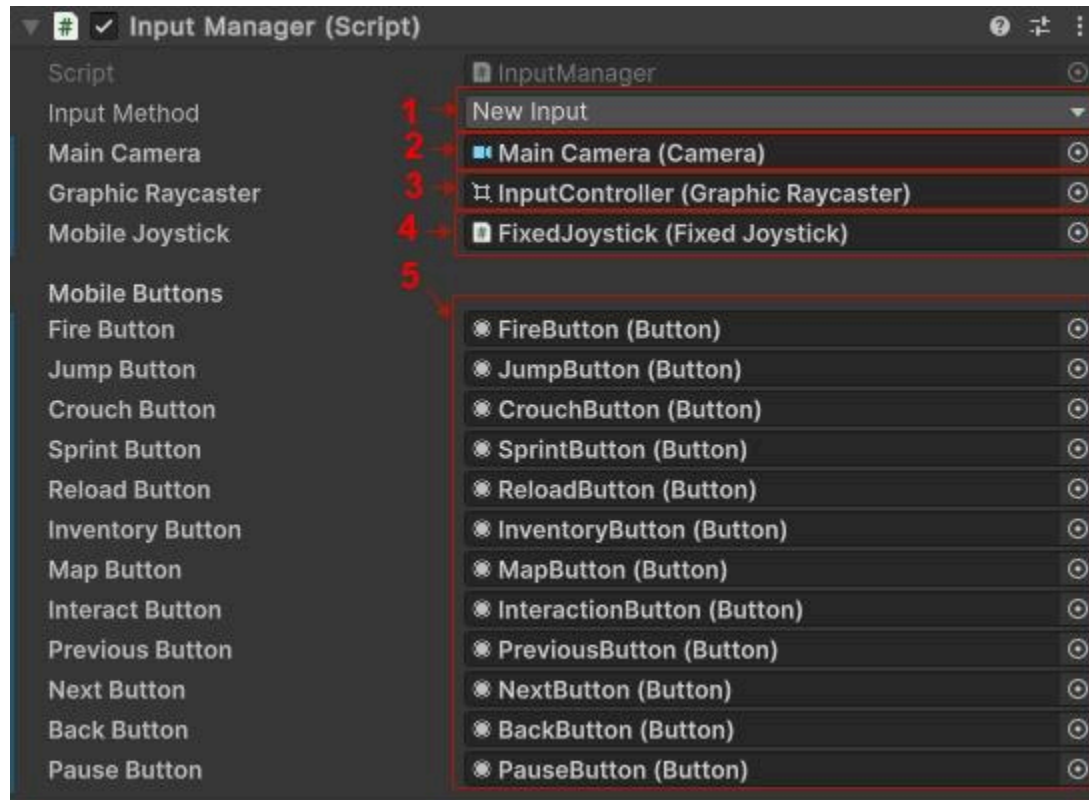


Once installed, you can find the **Input Kit** in the Unity editor under the **Assets** folder.

Navigate to **Input Kit > Prefabs > InputManager**, then drag and drop the **InputManager** prefab into your scene hierarchy.

How to Use

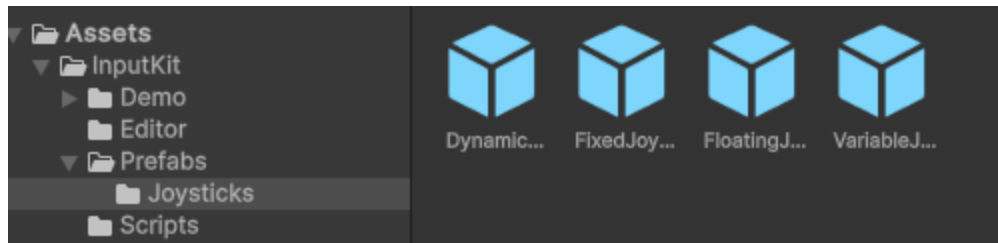
Input Manager Prefab



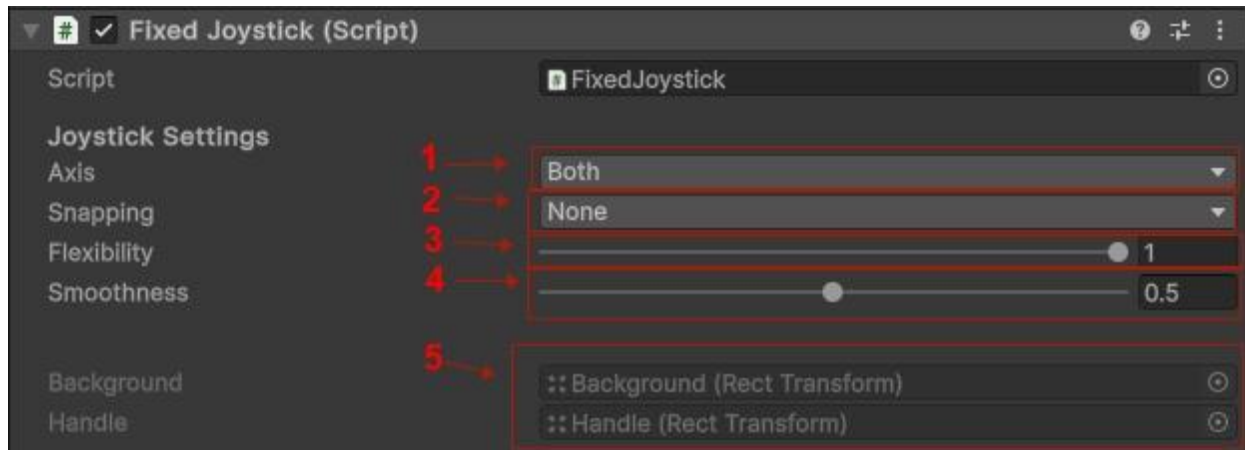
- 1 - Choose whether you want to use the **New Input System** or the **Legacy (Old) Input System** for your project.
- 2 - Drag and drop the **Main Camera** into your scene hierarchy. This is necessary for enabling raycasting functionality.
- 3 - To detect touch or mouse clicks on UI elements, drag and drop your **Canvas (with a Graphic Raycaster component)** into the designated field.
- 4 - To use any of the joystick prefabs included in the tool, make sure to drag and drop your chosen joystick prefab into the corresponding field.
- 5 - If your game uses mobile UI buttons, drag and drop the buttons you plan to use into the designated fields here.

Joysticks

Input Kit includes four types of premade joysticks: "Fixed", "Dynamic", "Floating", and "Variable". You can find them in the **Assets > InputKit > Prefabs > Joysticks** folder. To use them, drag and drop the desired joystick into your **Canvas** object in the hierarchy.



Joystick Prefab



- 1** - You can select which axis the joystick should respond to: horizontal, vertical, or both. This allows you to customize the joystick behavior based on your game's input needs.
- 2** - You can enable snapping to limit movement to four or eight directions. This feature helps align the character's movement with specific angles, ideal for grid-based or stylized directional control.
- 3** - Flexibility lets you control how far the joystick handle can move. You can set whether it stays within the joystick background or extends beyond it, giving you full control over the input range.
- 4** - Smoothness controls how closely the character's movement follows the joystick input. Higher smoothness results in more fluid and responsive movement, while lower values can make it feel more direct or snappy.
- 5** - The background and handle areas of the joystick are predefined in the prefab setup and cannot be reassigned manually.

Input Handler Script

Here is a full explanation of each method in the InputHandler class, along with example usage.

GetFire

Returns true if the fire/shoot input is pressed.

```
if (InputHandler.GetFire())
```

```
{  
    Debug.Log("Fire button pressed");  
}
```

GetJump

Returns true if the jump input is pressed.

```
if (InputHandler.GetJump())  
{  
    Debug.Log("Jump button pressed");  
}
```

GetCrouch

Returns true if the crouch input is pressed.

```
if (InputHandler.GetCrouch())  
{  
    Debug.Log("Crouch button pressed");  
}
```

GetSprint

Returns true if the sprint input is pressed.

```
if (InputHandler.GetSprint())  
{  
    Debug.Log("Sprint button pressed");  
}
```

GetReload

Returns true if the reload input is pressed.

```
if (InputHandler.GetReload())
```

```
{  
    Debug.Log("Reload button pressed");  
}
```

GetInventory

Returns true if the inventory input is pressed.

```
if (InputHandler.GetInventory())  
{  
    Debug.Log("Inventory button pressed");  
}
```

GetInteraction

Returns true if the interact input is pressed.

```
if (InputHandler.GetInteract())  
{  
    Debug.Log("Interact button pressed");  
}
```

GetMap

Returns true if the map input is pressed.

```
if (InputHandler.GetMap())  
{  
    Debug.Log("Map button pressed");  
}
```

GetPrevious

Returns true if the previous input is pressed.

```
if (InputHandler.GetPrevious())
```

```
{  
    Debug.Log("Previous button pressed");  
}
```

GetNext

Returns true if the next input is pressed.

```
if (InputHandler.GetNext())  
{  
    Debug.Log("Next button pressed");  
}
```

GetBack

Returns true if the back input is pressed.

```
if (InputHandler.GetBack())  
{  
    Debug.Log("Back button pressed");  
}
```

GetPause

Returns true if the pause input is pressed.

```
if (InputHandler.GetPause())  
{  
    Debug.Log("Pause button pressed");  
}
```

GetLeft

Returns true if the left button is pressed.



```
if (InputHandler.GetLeft())
{
    Debug.Log("Left button pressed");
}
```

GetRight

Returns true if the right button is pressed.

```
if (InputHandler.GetRight())
{
    Debug.Log("Right button pressed");
}
```

GetSwipe

Returns a directional enum (None, Left, Right, Up, Down) based on swipe gesture input.

```
if (InputHandler.GetSwipe() == InputHandler.SwipeLeft)
{
    Debug.Log("Swipe left detected");
}

if (InputHandler.GetSwipe() == InputHandler.SwipeRight)
{
    Debug.Log("Swipe right detected");
}
```

GetMove

Returns a Vector2 representing directional input (WASD, joystick, etc.).

```
if (InputHandler.GetMove() != Vector2.zero)
{
    Debug.Log("Movement detected: " + InputHandler.GetMove());
}
```

GetScreenPosition

Returns the input's position in screen coordinates (e.g., mouse or touch location).

```
if(InputHandler.GetScreenPosition() != Vector2.zero)
{
    Debug.Log("Screen position detected: " +
        InputHandler.GetScreenPosition());
}
```

GetWorldPosition

Returns the input's position in world coordinates (e.g., mouse or touch location).

```
if(InputHandler.GetWorldPosition() != Vector2.zero)
{
    Debug.Log("World position detected: " +
        InputHandler.GetWorldPosition());
}
```

GetCast2D

Performs a 2D raycast from the pointer and returns a Collider2D hit.

```
Collider2D hit2D = InputHandler.GetCast2D();
if (hit2D != null) Debug.Log("2D Object hit: " + hit2D.name);
```

GetCast3D

Performs a 3D raycast and returns a Collider hit.

```
Collider hit3D = InputHandler.GetCast3D();
if (hit3D != null) Debug.Log("3D Object hit: " + hit3D.name);
```

GetCastRect

Raycasts UI elements and returns a RectTransform if something was hit.

```
RectTransform uiHit = InputHandler.GetCastRect();
if (uiHit != null) Debug.Log("UI Element hit: " + uiHit.name);
```

GetKey<T>

Fetches a custom input value (bool, float, Vector2, etc.) by action name defined in the Input Actions asset.

```
float customFloat = InputHandler.GetKey<float>("CustomFloat");  
Vector2 customVector2 =  
InputHandler.GetKey<Vector2>("CustomVector2");  
bool customBool = InputHandler.GetKey<bool>("CustomBool");
```

Support

For any questions or support, please reach out to our support team at kirnazdogan@gmail.com